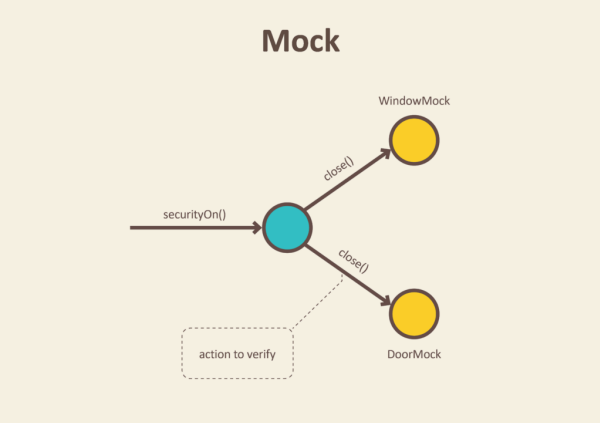
Laborator 06 - Unit Testing 2

Mocking



Un *mock object* inlocuieste si imita un obiect real din environmentul de testare. Este o unealta versatila si practica pentru imbunatatirea testelor.

Un motiv pentru a folosi *Python mock* este controlul comportamentului codului. De exemplu, daca in cadrul codului se realizeaza un request HTTP catre un serviciu extern, atunci testele au un comportament predictibil atata timp cat serviciul se comporta asa cum te astepti. Uneori, o schimbare temporara a comportamentului acestor servicii externe poate cauza esecuri intermitente in suita de testare. Din acest motiv, este mai bine ca testarea sa fie realizata intr-un mediu controlat. Inlocuirea request-ului cu un obiect*mock* oermite simularea ambelor situatii intr0un mod predictibil.

Uneori este dificila testarea unor zone de cod ce contin blocuri de tip exception sau ramuri ale unor if-uri complexe. Prin utilizarea unui *mock object* se poate ghida executia codului pentru a se testa anumite zone de cod.

Alt motiv pentru a utiliza un *mock object* este pentru a intelege mai bine comportamentul real al codului. Un obiect Python contine date despre modul in care a fost utilizat si care pot fi inspectate:

- daca o metoda a fost apelata
- cum a fost apelata o metoda
- cat de des a fost apelata o metoda

Biblioteca Python Mock

Biblioteca **unittest.mock** ofera o clasa, *Mock*, care poate fi utilizata pentru imitarea obisctelor reale din cod. *Mock* ofera fiexibilitatea si datele specifice si, impreuna cu toate subclasele, satisface majoritatea nevoilor pentru *Python mocking*. In plus, biblioteca ofera o functie, *patch()*, care inlocuieste obiectul real in cod cu o instanta *Mock*. Se poate utiliza *patch()* drept un manager de context, oferind control asupra modulului in care alte obecte vor fi imitate.

Mock Object

unittest.mock ofera o clasa de baza pentru imitarea obiectelor numita **Mock**. Flexibilitatea acestei clase ofera ofera un numar nelimitat de utilizari.

Analiza valorilor de return

Un motiv pentru a utiliza obiecte de tip *mock* este acela de a controla comporatamentul codului in timpul testarii. Un mod in care se poate face acest lucru este prin specificarea valorilor de return ale unor functii.

Functia din aceasta secventa de cod are rolul de a testat daca azi este o zi lucratoare (luni-vineri). In cazul in care functia este testata in weekend, o sa se afieseze *AssertionError* chiar daca functia este scrisa corect.

```
from datetime import datetime

def is_weekday():
    today = datetime.today()
    # Python's datetime library treats Monday as 0 and Sunday as 6
    return (0 <= today.weekday() < 5)

# Test if today is a weekday
assert is_weekday()
```

Este nevoie sa garantam comportamentul predictibil al testelor. Se poate utiliza *Mock* pentru a elimina incertitudinea din cod in timpul testarii. In acest caz, imitam comportamentul functiei *datetime* prin setarea valorii *.return_value()* pentru *.today()* la o zi alesa..

```
import datetime
from unittest.mock import Mock

# Save a couple of test days
tuesday = datetime.datetime(year=2019, month=1, day=1)
saturday = datetime.datetime(year=2019, month=1, day=5)

# Mock datetime to control today's date
datetime = Mock()

def is_weekday():
    today = datetime.datetime.today()
    # Python's datetime library treats Monday as 0 and Sunday as 6
    return (0 <= today.weekday() < 5)

# Mock .today() to return Tuesday
datetime.datetime.today.return_value = tuesday
# Test Tuesday is a weekday
assert is_weekday()
# Mock .today() to return Saturday
datetime.datetime.today.return_value = saturday
# Test Saturday is not a weekday
assert not is_weekday()
```

In primul caz, se garanteaza ca *tuesday* este o zi lucratoare, iar in al doilea caz se testeaza ca *saturday* nu este zi lucratpare. Astfel, nu mai conteaza cand este tesata functia.

Cand construim teste, existra cazuri cand modificare valorii de return nu este suficienta intrucat unele functii sunt mai complicate sau nu au un singur fir de exectie. Uneori se doreste sa se apeleze functia de mai multe opri, cu valori de return diferite sau chiar cu sa ridice exceptii. Pentru asta se poate utiliza *.side_effect*.

Efecte laterale

Putem controla comportamentul codului prin specificarea efectelor laterale ale unei functii imitate. *.side_effect* defineste ce se intampla cand apelam o functie imitata.

Functia *get_holidays()* face un request la un server *localhost* pentru a obtine un set de vacante. In cazul in care serverul raspunde cu succes, functia returneaza dictionarul. Altfel, se returneaza *None*.

```
import requests

def get_holidays():
    r = requests.get('http://localhost/api/holidays')
    if r.status_code == 200:
        return r.json()
    return None
```

Putem testa cum functia *get_holidays()* raspunde la un *connection timeout* prin setarea *requests.get.side_effect*. Utilizam *.assertRasises()* pentru a verifica daca *get_holidays()* ridica o exceptie luand in considerare efectele laterale.

```
import unittest
from requests.exceptions import Timeout
from unittest.mock import Mock

# Mock requests to control its behavior
requests = Mock()

def get_holidays():
    r = requests.get('http://localhost/api/holidays')
    if r.status_code == 200:
        return r.json()
    return None

class TestCalendar(unittest.TestCase):
    def test_get_holidays_timeout(self):
        # Test a connection timeout
        requests.get.side_effect = Timeout
        with self.assertRaises(Timeout):
```

```
            get_holidays()

if __name__ == '__main__':
    unittest.main()
```

patch()

unittest.mock pune la dispozitie un mecanism puternic pentru imitarea obiectelor, *patch()*, care inlocuieste toate aparitiile unui obiect dat cu un *Mock* intr-un modul.

patch() este folosit in doua moduri:

- *decorator*
- *context manager*

Decorator

Pana acum am *monkey patched* obiecte in fisierul principal. *Monkey patching* presupune inlocuirea unui obiect cu altul al *runtime*. Acum vom folosi *patch()*pentru a inlocui obiectele.

Pastram in fisierul *my_calendar.py* functiile *is_weekend()* si *get_holidays()*, iar in fisierul *tests.py* adagam testele noi. *patch()* primeste calea catre obiectul care trebuie imitat.

- my_calendar.py

```
import requests
from datetime import datetime

def is_weekday():
    today = datetime.today()
    # Python's datetime library treats Monday as 0 and Sunday as 6
    return (0 <= today.weekday() < 5)

def get_holidays():
    r = requests.get('http://localhost/api/holidays')
    if r.status_code == 200:
        return r.json()
    return None
```

```
import unittest
from my_calendar import get_holidays
from requests.exceptions import Timeout
from unittest.mock import patch

class TestCalendar(unittest.TestCase):
    @patch('my_calendar.requests')
    def test_get_holidays_timeout(self, mock_requests):
            mock_requests.get.side_effect = Timeout
            with self.assertRaises(Timeout):
                get_holidays()
                mock_requests.get.assert_called_once()

if __name__ == '__main__':
    unittest.main()
```

In acest caz, am putut folosi *patch()* drept *decorator*. In alte cazuri, *patch()* poate fi folosit drept *context manager*.

Context manager

Cateva motive pentru a utiliza *patch()* drept *context manager* sunt urmatoarele:

- dorim sa imitam un obiect doar pentru o parte din test
- folosim deja prea multe instante de tip *decorator* sau parametrii, care pot ingreuna citirea codului

Pentru aceasta situatie, folosim un *with statement*:

```
import unittest
from my_calendar import get_holidays
```

```
from requests.exceptions import Timeout
from unittest.mock import patch

class TestCalendar(unittest.TestCase):
    def test_get_holidays_timeout(self):
        with patch('my_calendar.requests') as mock_requests:
            mock_requests.get.side_effect = Timeout
            with self.assertRaises(Timeout):
                get_holidays()
                mock_requests.get.assert_called_once()

if __name__ == '__main__':
    unittest.main()
```

La finalizarea *with*-ului, *patch()* inlocuieste obiectul imitat cu cel original.

Pana acum am imitat obiecte complete, dar uneori avem nevoie sa imitam doar o parte dintr-un obiect.

Patching an Object's Attributes

Daca dorim sa inlocuim doar o metoda din cadrul unui obiect in locul obiectului complet, putem sa utilizam *patch.object()*.

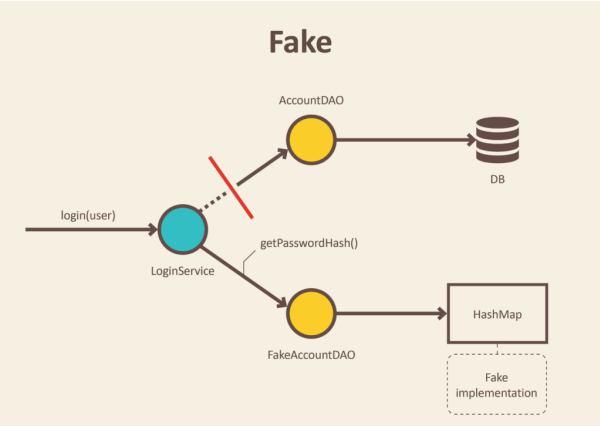
```
import unittest
from my_calendar import requests, get_holidays
from unittest.mock import patch

class TestCalendar(unittest.TestCase):
    @patch.object(requests, 'get', side_effect=requests.exceptions.Timeout)
    def test_get_holidays_timeout(self, mock_requests):
            with self.assertRaises(requests.exceptions.Timeout):
                get_holidays()

if __name__ == '__main__':
    unittest.main()
```

In acest exemplu, inlocuim doar *get()* in loc de toata instanta *requests*. Toate celelalte atribute raman la fel. *object()* primeste minim doi parametrii: obiectul care trebuie imitat, atributul pe care dorim sa il imitam.

Fakes



Un obiect *fake* implementeaza aceeasi interfata ca obiectul real, incluzand toate metodele si atributele originale. Insa, implementarea acestor metode poate presupune cateva scurtaturi pentru a imita functionalitatile care nu sunt cu adevarat necesare pentru a produce un rezultat valid.

Un exemplu, este un obiect care realizeaza un calcul complex ce poate fi inlocuit cu unul *fake* ce ce include date precalculate care pot fi returnate rapid fara a reliza cu adevarat calculele.

Daca avem nevoie de un obiect responsabil de stocarea datelor intr-o baza de date, putem crea o versiune *fake* care poate stoca datele intr-un *hash map*. O sa putem stoca si analiza obiect, dar in loc sa folosim o baza de date reala cu un numar mare de intrari, o sa fie nevoie sa stocam un numar mic de intrari care poate fi incarcat cu usurinta la testare. Datele pot fi generate cu usurinta prin interfediul bibliotecii *Faker*.

Faker

Faker este o biblioteca in Python care genereaza date false. Este des folosita pentru testare sau pentru umplerea unei baze de date cu date *dummy*.

Generarea de date simple

Biblioteca *Faker* pune la dispozitie un numar mare de metode predefinite cu care putem genera date: *name*, *first_name*, *last_name*, *address*, *job*, *city*, *latitude*, *logitude*, *text*, *word*, *date_of_birth*, *timezone*, *random_int*

etc.

```
from faker import Faker
fake = Faker()

fake.name()
# 'Lucy Cechtelar'

fake.address()
# '426 Jordy Lodge
#  Cartwrightshire, SC 88120-6700'

fake.text()
# 'Sint velit eveniet. Rerum atque repellat voluptatem quia rerum. Numquam excepturi
#  beatae sint laudantium consequatur. Magni occaecati itaque sint et sit tempore. Nesciunt
#  amet quidem. Iusto deleniti cum autem ad quia aperiam.'

for _ in range(10):
  print(fake.name())

# 'Adaline Reichel'
# 'Dr. Santa Prosacco DVM'
# 'Noemy Vandervort V'
# 'Lexi O'Conner'
# 'Gracie Weber'
# 'Roscoe Johns'
# 'Emmett Lebsack'
# 'Keegan Thiel'
# 'Wellington Koelpin II'
# 'Ms. Karley Kiehn V'
```

Generarea de profiluri

In exemplele anterioare, am generat tipuri diferite de date cum ar fi varsta, nume, adresa, etc, dar le-am generat separat. *Faker* pune la dispozitie o metoda care genereaza profiluri complete.

```
from faker import Faker
import dumper

faker = Faker()

profile1 = faker.simple_profile()
dumper.dump(profile1)

#  username: 'gmorgan'
#  name: 'Jessica Clark'
#  sex: 'F'
#  address: '694 Joseph Valleys\nJohnfort, ME 81780'
#  mail: 'bettybuckley@yahoo.com'
#  birthdate: <str at 0x20d5bcbd7b0>: 'datetime.date(1938, 9, 18)'

profile2 = faker.simple_profile('M')
dumper.dump(profile2)
#  username: 'mrios'
#  name: 'Harold Wagner'
#  sex: 'M'
#  address: '26439 Robinson Radial\nWest Meghanmouth, PA 85463'
#  mail: 'josechoi@gmail.com'
#  birthdate: <str at 0x20d5bcbd7b0>: 'datetime.date(1986, 8, 18)'

profile3 = faker.profile(sex='F')
dumper.dump(profile3)
# job: 'Engineer, communications'
#  company: 'Jackson-Willis'
#  ssn: '430-41-6118'
#  residence: 'USNS Odom\nFPO AP 47095'
#  current_location: <tuple at 0x20d5bca9a88>
#    0: <str at 0x20d5bd248a0>: "Decimal('74.1885785')"
#    1: <str at 0x20d5bd248a0>: "Decimal('119.951995')"
#  blood_group: 'O-'
#  website: ['http://roberson.com/']
#  username: 'ygutierrez'
#  name: 'Lindsay Walker'
#  sex: 'F'
#  address: '22968 Beverly Road Suite 918\nTimothyborough, SD 10494'
#  mail: 'aliciamccall@yahoo.com'
#  birthdate: <str at 0x20d5bcbd7b0>: 'datetime.date(1979, 1, 4)'
```

Generare unor date unice

Pentru toate exemplele de mai sus, obtinem rezultate diferite pentru executii multiple. Insa, nu exista gatantia ca datele generate sunt unice in cazul unor bucle de oridinul miilor. Pentru a depasi acest obstacol. biblioteca *Faker* pune la dispozitie metoda *unique* ce garanteaza unicitatea datelor generate in acelasii context sau in aceeasi bucla.

```
names = [fakeobject.unique.name() for i in range(10)]
for i in range (0,len(names)):
  print(names[i])
# 'Adaline Reichel'
# 'Dr. Santa Prosacco DVM'
# 'Noemy Vandervort V'
# 'Lexi O'Conner'
# 'Gracie Weber'
# 'Roscoe Johns'
# 'Emmett Lebsack'
# 'Keegan Thiel'
# 'Wellington Koelpin II'
# 'Ms. Karley Kiehn V'
```

In anumite cazuri, poate fi necesara regenerarea acelorasi date in mai multe parti ale codului, prin intermediul metodei *seed()*.

```
Faker.seed(10)
print(fakeobject.name())
```

Astfel, se garanteaza ca o sa existe 10 nume diferite care se pot repeta. Daca apelam de minim 11 ori functia *fakeobject.name()* se garanteaza ca va exista o minim o pereche de rezultate identice.

Localizare

faker.Faker poate primi ca argument un ID local (LCID). Daca nu este oferit un ID local, se foloseste setarea standard *en_US*.

```
from faker import Faker
fake = Faker('it_IT')
for _ in range(5):
    print(fake.name())

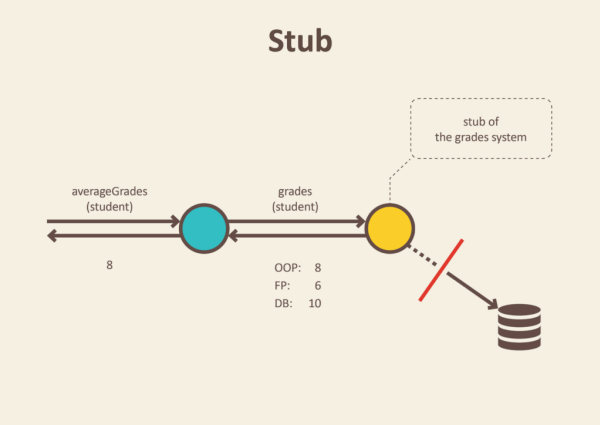
# 'Elda Palumbo'
# 'Pacifico Giordano'
# 'Sig. Avide Guerra'
# 'Yago Amato'
# 'Eustachio Messina'

from faker import Faker
fake = Faker(['it_IT', 'en_US', 'ja_JP'])
for _ in range(10):
    print(fake.name())

# 鈴木 陽一
# Leslie Moreno
# Emma Williams
# 渡辺 裕美子
# Marcantonio Galuppi
# Martha Davis
# Kristen Turner
# 中津川 春香
# Ashley Castillo
# 山田 桃子
```

Stubs

Stub-urile sunt obiecte care returneaza valori predefinite. Un *stub* este un obiect care seamana cu cel original dar care are doar metodele necesare pentru testare. Cand nu dorim sa utilizam obiectele care pot sa raspunda cu datele reale, folosim *stub*-uri.



Putem implementa interfete care ne poate separa de o biblioteca *third-party* daca dezvoltam parte de *back-end* a unei aplicatii care va interactiona cu un API. Acea interfata va produce valori *hard-coded* si va servi drept *stub*.

In cazul in care testam o singura aplicatie, putgem crea un *stub* pentru *Rest API*-urile pe care se bazeaza.

```
temperature = ThermometerRead(Outside)
if temperature > 40:
    print("It is hot!")
```

In acest caz nu avem acces la datele reale, asa ca folosim un *stub* care inlocuieste functia *ThermometerRead()*.

```
def ThermometerRead(temp)
    return 28
```

Exercitii

Setup

```
pip install mock
pip install Faker
```

0. Clonati repo-ul [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/unit-testing-2-lab].

1. Testati in minim 2 moduri functia *is_leap_year*, folosind *mock*.

2. Folosind *patch decorator*, testati functia *get_marks* in cazul in care conexiunea da *timeout*.

3. Folosind *patch context manager*, implementati functia de teste pentru *calculate_total*.

Pentru cerintele 4 si 5 puteti folosii urmatorul scenariu:

O firma de publicitate doreste sa afle ce persoane sunt interesate de produsele firmei FirmaMistoSRL. Pentru asta, au creat un formular online pe care sa il completeze utilizatorii de pe platformele de socializare. Ajutati echipa de IT a firmei de publicitate sa testeze programul ce analizeaza rezultatele formularului.

4. Creati o baza de date *fake* ce contine minim 500 de intrari. O intrare este reprezentata de o instanta a clasei Person (pe care trebuie sa o creati) care are minim 3 atribute (nume, varsta, email). Cheia unica este representata de adresa de mail.

5. Creati un *stub* si o functie *fake* care sa simuleze functionalitatea metodei *get(cheie_unica)* pe o baza de date. Puteti sa folositi *baza de date* creata anterior (ex. get_youngest_person, get_new_person)

tsc/laboratoare/laborator-06.txt · Last modified: 2024/02/29 14:40 (external edit)